

ЛАБОРАТОРНАЯ РАБОТА №1.
СОЗДАНИЕ ВИЗУАЛЬНОГО КОМПОНЕНТА
СОДЕРЖАНИЕ

Цель	2
Задание	2
Создание визуального компонента	3
Блоки визуальных компонент.....	9
Варианты.....	19

Цель

Научится создавать визуальные компоненты и встраивать их в другие проекты.

Визуальные компоненты применяются с целью уменьшения количества кода (если часть логики идет в компоненте и потом используется в других компонентах или формах, например, вывод списка значений перечисления в выпадающий список с возможностью получения выбранного значения и использование этого выпадающего списка в нескольких формах) или для разработки сложных компонент (состоит из нескольких взаимосвязанных компонент, например, DataGridView с кнопками для вызова форм добавления, редактирования и удаления записей). Также, компоненты, с расширенным функционалом и дизайном по сравнению со стандартными компонентами, включены в наборы платных библиотек (например, devexpress). Разработка компонент является как бы более глубоким уровнем программирования, так как пользователями данной продукцией будут не конечные пользователи, как при разработке, например, коммерческих приложений, а программисты, которые будут использовать эти компоненты в своих приложениях.

Задание

В рамках первой лабораторной вам предстоит выполнить следующие задачи:

1. *Создать библиотеку, в которой реализовать визуальные компоненты (3 штуки) по заданию.*
2. *Создать описание компонентов, какие есть методы, события, свойства у компонент и как их использовать (будет включено в итоговый отчет).*
3. *Реализовать десктопное приложение (Windows Forms), для проверки работоспособности компонент.*

Все компоненты разбиты условно на 3 блока (выбора, ввода значения и вывода списка). В каждом блоке представлено по 3 различных варианта

используемых компонент или 3 варианта работы с компонентом. Также для некоторых компонент имеются 3 варианта загрузки данных, необходимых для их работы.

Создание визуального компонента

Рассмотрим пример, как создавать визуальные компоненты. Компонент будем создавать сразу в отдельной библиотеке, чтобы потом его можно было бы «прикручивать» в различные проекты. Пример будет реализован в среде Visual Studio 2022, на платформе .Net 6.

Первым делом нужно будет создать библиотеку, но не простую, а с поддержкой Windows Forms (рисунок 1.1).

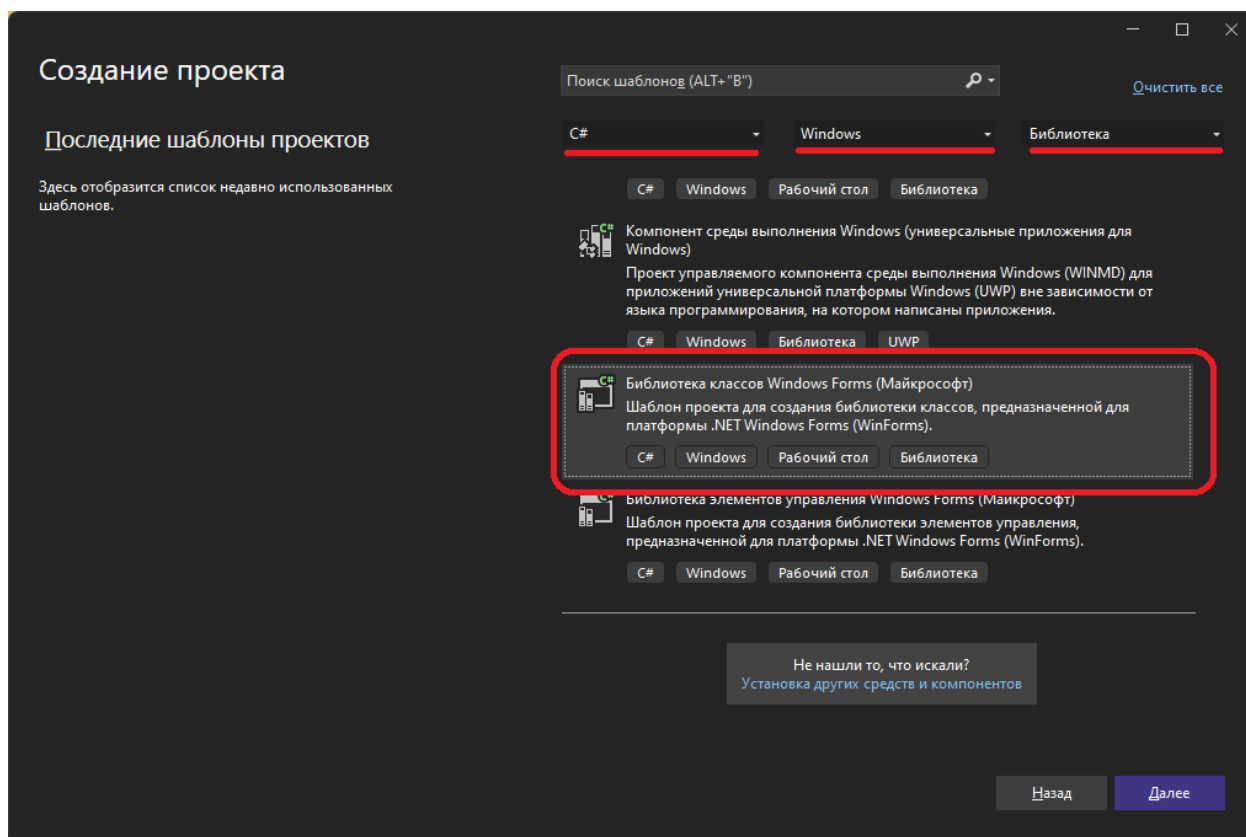


Рисунок 1.1 – Выбор типа библиотеки

Далее в проекте создадим «Пользовательский элемент управления» (рисунок 1.2). Для этого кликаем правой кнопкой мыши по имени проекта, выбираем в появившемся меню пункт «Добавить» и в нем пункт

«Пользовательский элемент управления (Windows Forms)...». В появившемся окне задаем имя для нашего визуального компонента.

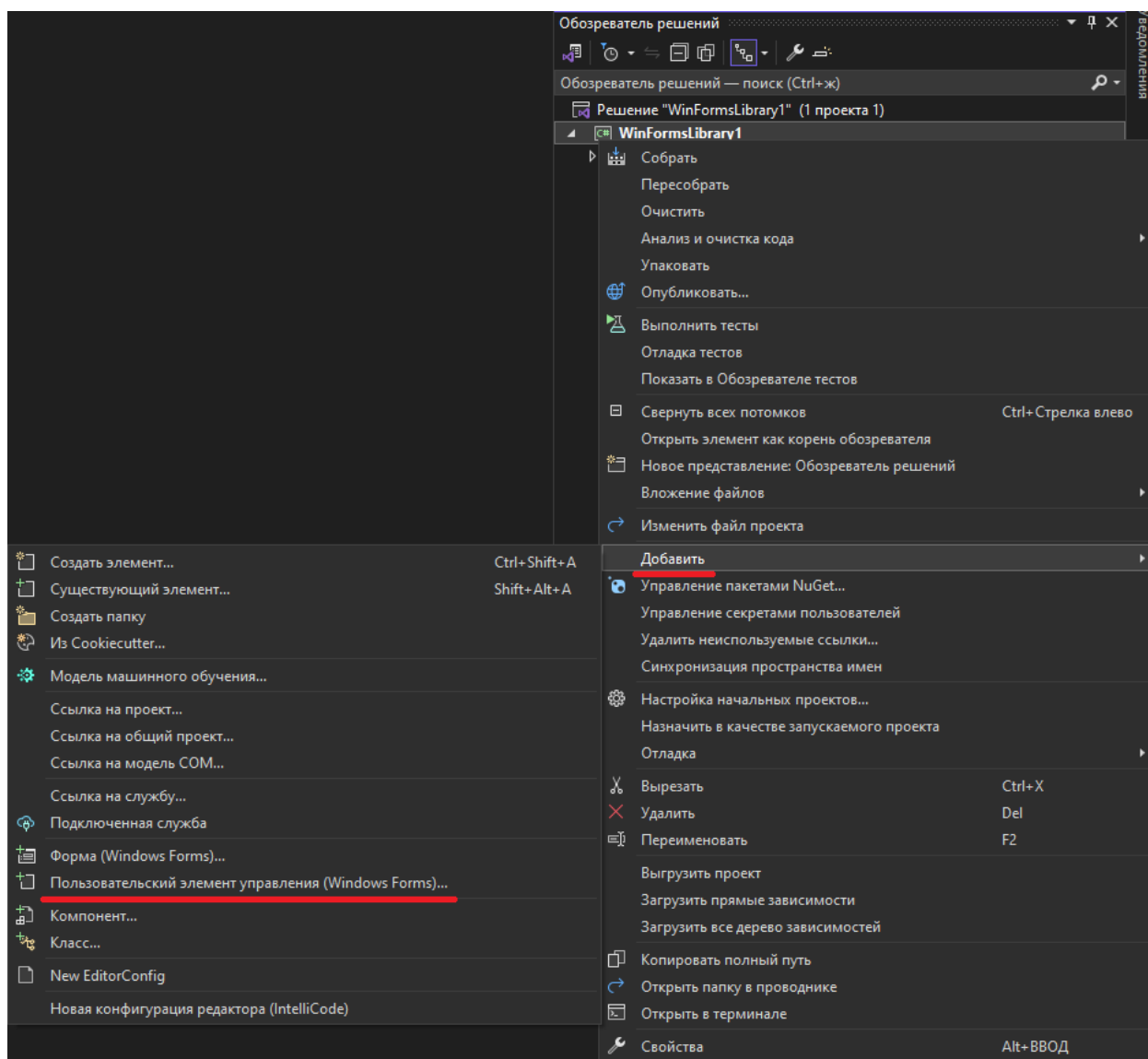


Рисунок 1.2 – Добавление нового элемента

В результате в проект добавляется новый элемент, очень похожий по своей структуре с формой. Он также как и форма разделен физически на несколько файлов (в частности, отдельно идет код, отвечающий за дизайн контрола и отдельно код с логикой). Также имеется дизайнер формы, через который можно настроить внешний вид контрола, который повторяет дизайнер для работы с обычными формами (рисунок 1.3).

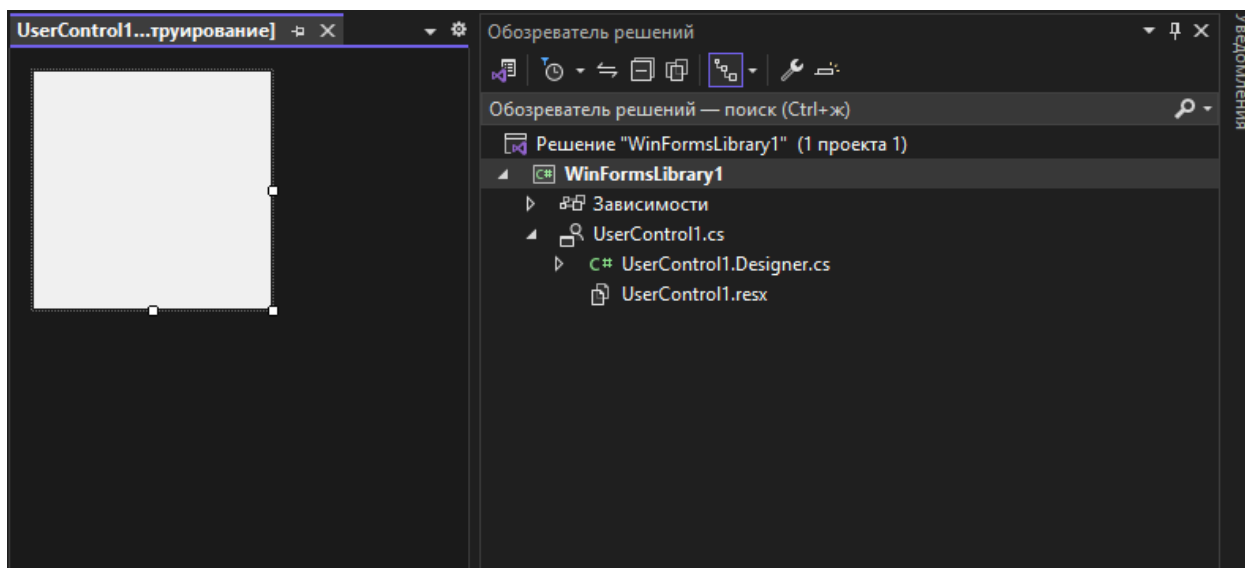


Рисунок 1.3 – Дизайнер контрола

Следующим шагом потребуется наполнить контрол элементами и логикой. Допустим, контрол объединит некую картинку и кнопку для загрузки картинки. Добавим на контрол элементы PictureBox и Button (рисунок 1.4). Сразу настроим для «красоты», что кнопка будет «привязана» только к нижней границе контрола (свойство Anchor у кнопки), а PictureBox «привяжем» ко всем краям контрола, чтобы он «растягивался» при изменении размеров контрола.

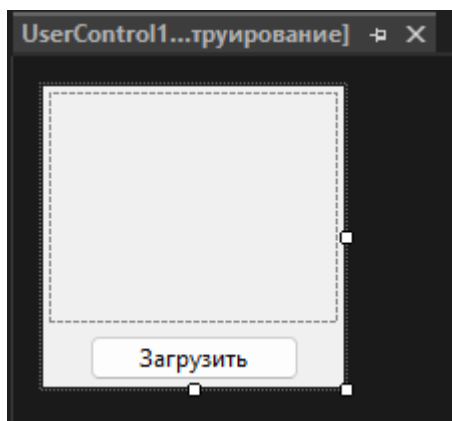


Рисунок 1.4 – Контрол с добавленными элементами

Далее перейдем в логику. Для кнопки при нажатии будет вызываться диалоговое окно выбора файла и, если файл выбран, попытка загрузить изображение из него в PictureBox. Также сделаем публичное свойство, чтобы можно было задать изображение в PictureBox, либо получить с него изображение. И, дополнительно, событие, чтобы можно было «реагировать»

при смене изображения. Для обработок ошибок (а у нас может быть ошибка при попытке загрузить изображение) введем поле с текстом ошибки, а также событие, которое будет вызываться, при возникновении ошибки (листинг 1.1).

```
namespace WinFormsLibrary1
{
    public partial class UserControl1 : UserControl
    {
        private event EventHandler? _avatarChanged;

        private event Action? _errorOccured;

        public string Error { get; private set; }

        public Image Avatar
        {
            get
            {
                return pictureBoxAvatar.Image;
            }
            set
            {
                pictureBoxAvatar.Image = value;
            }
        }

        public event EventHandler AvatarChanged
        {
            add { _avatarChanged += value; }
            remove { _avatarChanged -= value; }
        }

        public event Action AnErrorOccurred
        {
            add { _errorOccured += value; }
            remove { _errorOccured -= value; }
        }

        public UserControl1()
        {
            InitializeComponent();
            Error = string.Empty;
        }

        private void ButtonLoad_Click(object sender, EventArgs e)
        {
            var ofd = new OpenFileDialog();
            if (ofd.ShowDialog() == DialogResult.OK)
            {
                try
                {
                    pictureBoxAvatar.Image =
Image.FromFile(ofd.FileName);
                    _avatarChanged?.Invoke(this, e);
                }
                catch (Exception ex)
                {
                    Error = ex.Message;
                    _errorOccured?.Invoke();
                }
            }
        }
    }
}
```

```
}  
}
```

Листинг 1.1 – Код логики контрола

Далее, выполняем сборку проекта и в панели элементов в дизайнере форм видим свой новый пользовательский элемент (рисунок 1.5).

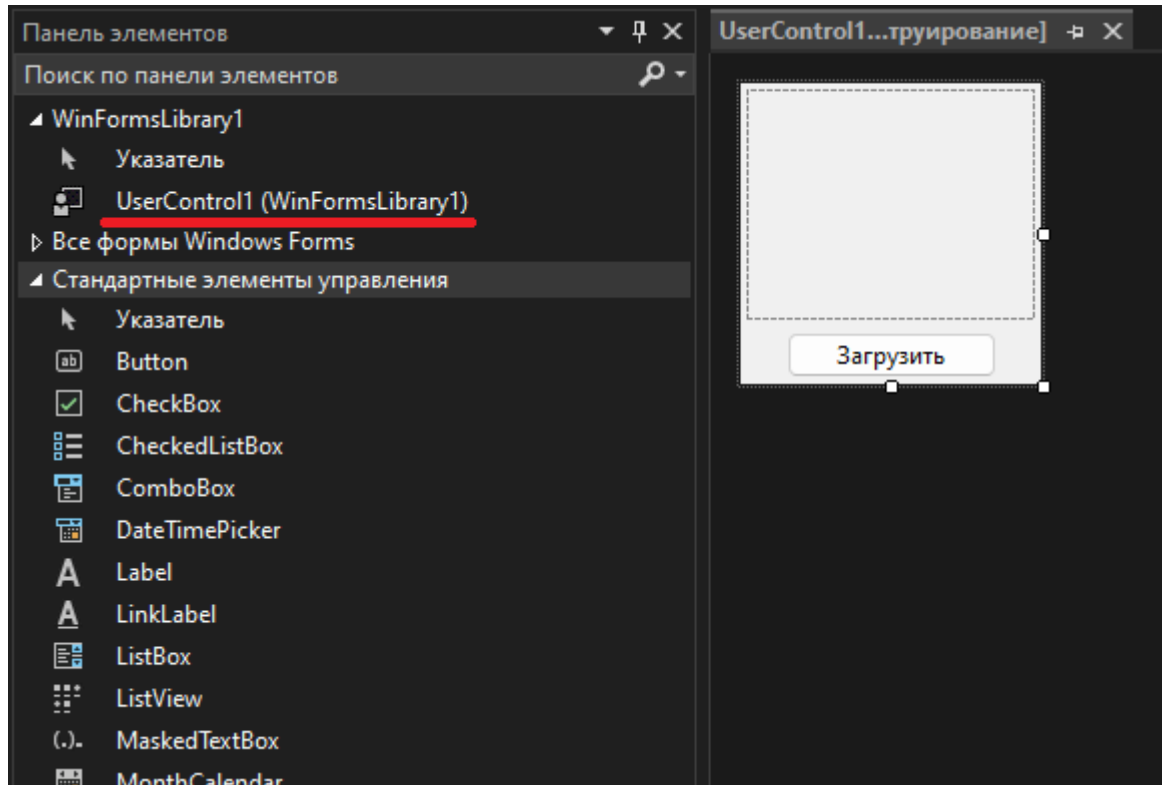


Рисунок 1.5 – Новый элемент в панели элементов

Для проверки можно сделать в рамках того же решения еще один проект с Windows Forms (уже не библиотеку, а исполняемый проект), на форму поместить этот элемент (пользовательские элементы в рамках одного решения должны свободно распространяться) и в логике проверить работу всех его свойств и событий, все отладить (рисунок 1.6, листинг 1.2).

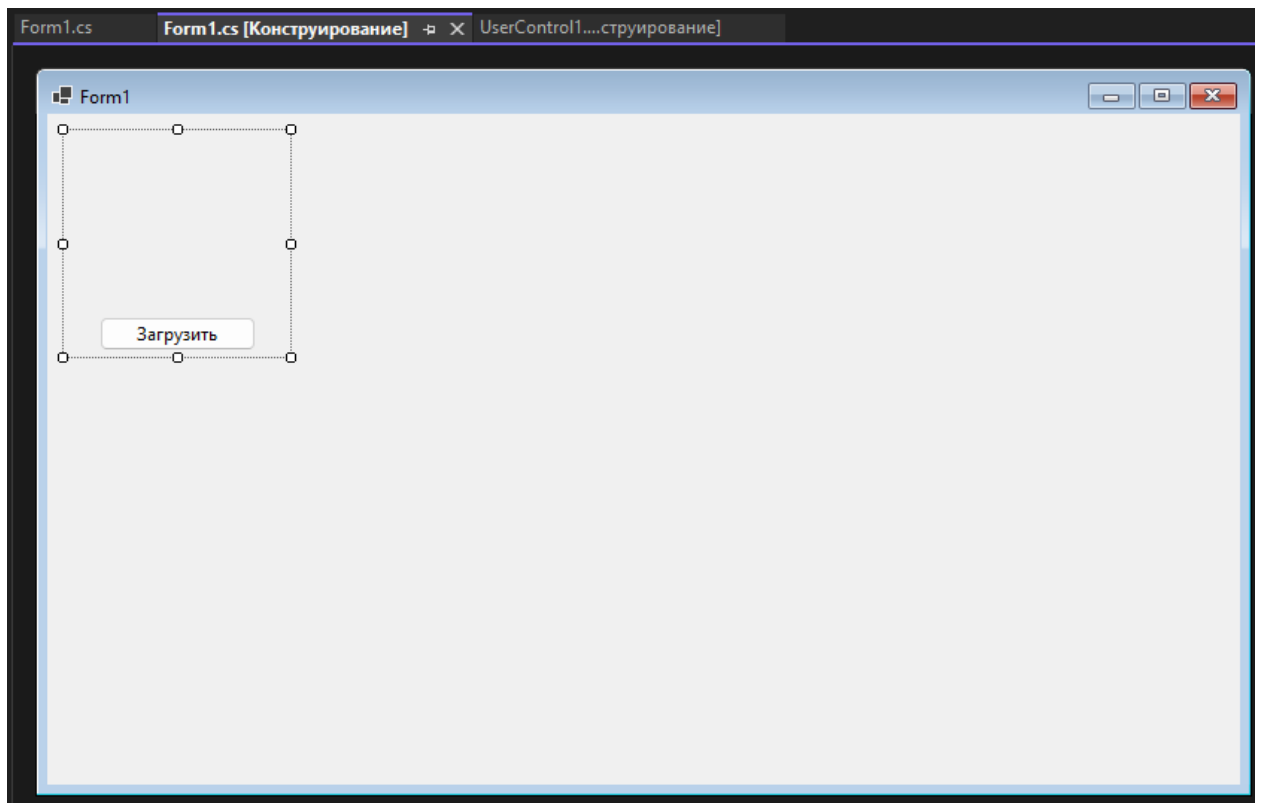


Рисунок 1.6 – Добавление контрола на форму

```
namespace WinFormsApp1
{
    public partial class Form1 : Form
    {
        public Form1()
        {
            InitializeComponent();
            CreateImage();
        }

        private void CreateImage()
        {
            Bitmap bmp = new(userControl11.Width - 10,
userControl11.Height - 10);
            Graphics gr = Graphics.FromImage(bmp);

            gr.DrawEllipse(new Pen(Color.Red), 10, 10, 20, 20);
            userControl11.Avatar = bmp;
        }

        private void userControl11_AvatarChanged(object sender, EventArgs e)
        {
            var width = userControl11.Avatar.Width;
            MessageBox.Show($"Change avatar, width={width}");
        }
    }
}
```

Листинг 1.2 – Код логики формы

Результат работы на рисунках 1.7-1.8.

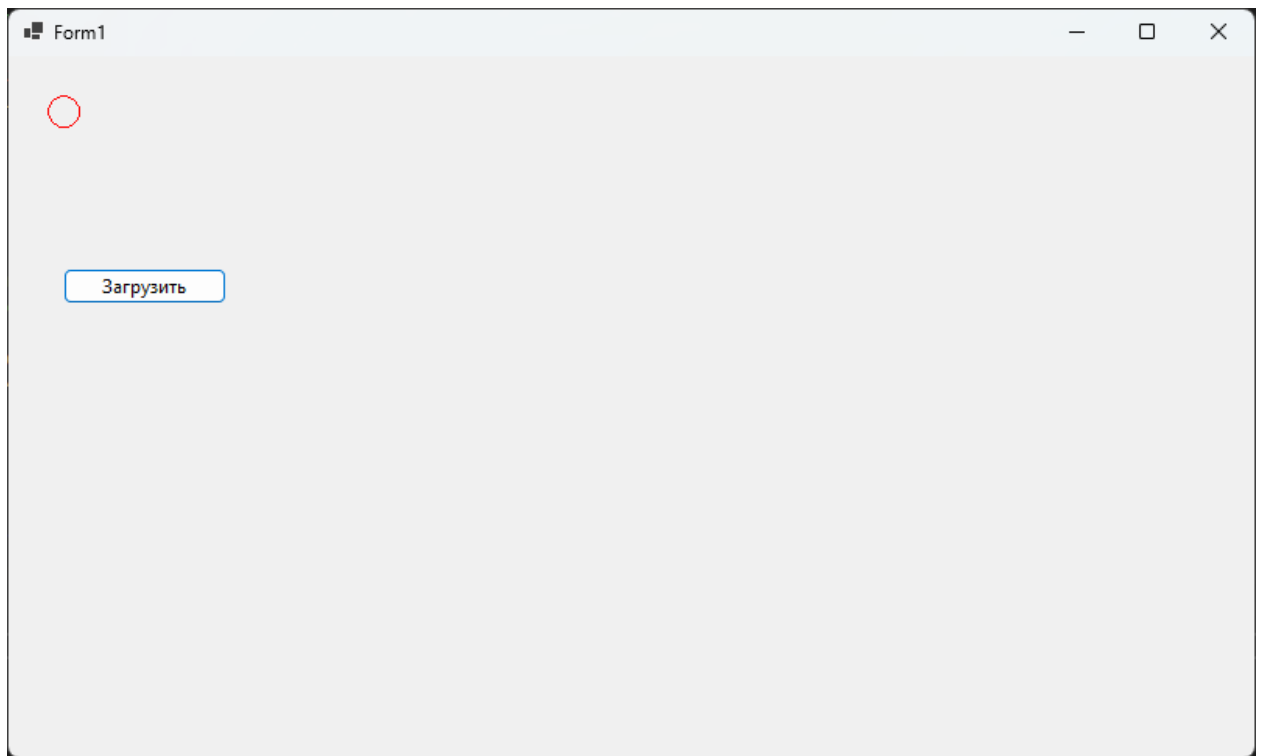


Рисунок 1.7 – Старт программы

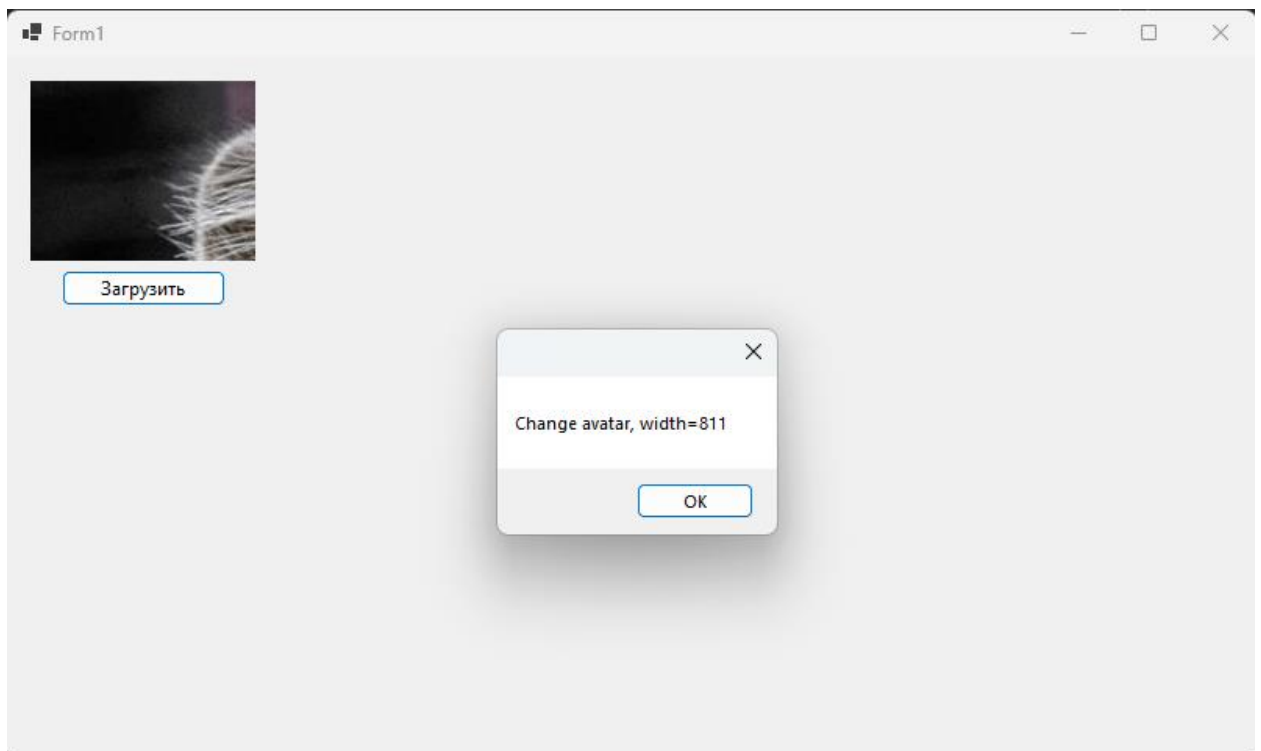


Рисунок 1.8 – Смена рисунка

Блоки визуальных компонент

- **Визуальный компонент выбора:**

- Визуальный компонент для выбора из выпадающего списка (ComboBox). Список заполняется строковыми значениями. Заполнение может происходить 3 возможными способами, в зависимости от варианта:

- через метод, у которого в передаваемых параметрах идет список строк и через этот список идет заполнение ComboBox;
- через метод, у которого в передаваемых параметрах идет строка, которая добавляется в Items элемента ComboBox;
- через публичное свойство, которое передает ссылку на свойство Items элемента ComboBox, через которое и идет заполнение.

Отдельный публичный метод очистки списка. Публичное свойство (set, get) для установки и получения выбранного значения (возвращает пустую строку, если нет выбранного значения), а также событие, вызываемое при смене значения в ComboBox.

- Визуальный компонент для выбора из списка с выбором (CheckedListBox). Список заполняется строковыми значениями. Заполнение может происходить 3 возможными способами, в зависимости от варианта:

- через метод, у которого в передаваемых параметрах идет список строк и через этот список идет заполнение CheckedListBox;
- через метод, у которого в передаваемых параметрах идет строка, которая добавляется в Items элемента CheckedListBox;
- через публичное свойство, которое передает ссылку на свойство Items элемента ComboBox, через которое и идет заполнение.

Отдельный публичный метод отчистки списка. Публичное свойство (set, get) для установки и получения выбранного значения (возвращает пустую строку, если нет выбранного значения), а также событие, вызываемое при смене значения в `CheckedListBox`.

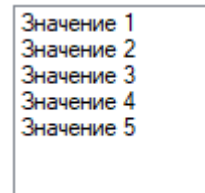
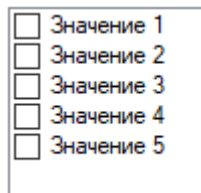
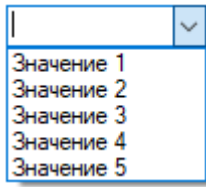
○ Визуальный компонент для выбора из обычного списка (ListBox).

Список заполняется строковыми значениями. Заполнение может происходить 3 возможными способами, в зависимости от варианта:

- через метод, у которого в передаваемых параметрах идет список строк и через этот список идет заполнение `ListBox`;
- через метод, у которого в передаваемых параметрах идет строка, которая добавляется в `Items` элемента `ListBox`;
- через публичное свойство, которое передает ссылку на свойство `Items` элемента `ListBox`, через которое и идет заполнение.

Отдельный публичный метод отчистки списка. Публичное свойство (set, get) для установки и получения выбранного значения (возвращает пустую строку, если нет выбранного значения), а также событие, вызываемое при смене значения в `ListBox`.

Пример. Имеется список значений: "Значение 1", "Значение 2", "Значение 3", "Значение 4", "Значение 5". Передаем список одним из способов в компоненты и получаем на форме заполненный выпадающий список (рисунок 1.9а), список с выбором (рисунок 1.9б) и обычный список (рисунок 1.9в).



а) выпадающий список б) список с выбором в) обычный список

Рисунок 1.9 – Результаты заполнения список с выбором значениями

- **Визуальный компонент ввода:**

- Текстовый визуальный компонент для ввода значения определенного типа (TextBox), предусматривающий возможность пустого (не заполненного значения). В контроле должен быть CheckBox, который при проставленной галочке будет обозначать признак незаполненного значения (null) и неактивным элементом ввода значения (TextBox). При не проставленной галочке элемент ввода значения должен быть активным для заполнения.

Публичное свойство для установки и получения введенного значения (set, get):

- при установке: выставить/убрать галочку у CheckBox, в зависимости от того, какое значение приходит, а в TextBox вывести или значение, или пусто;
- при получении: если у CheckBox не проставлена галочка, а в элементе не введено значение, выдавать ошибку (сделать свой класс ошибок). Если введенное значение не соответствует требуемому типу, выдавать ошибку (сделать свой класс ошибок).

Создать также событие, вызываемое при смене значения (либо при смене CheckBox).

- Текстовый визуальный компонент для ввода значения с проверкой по шаблону (TextBox). Шаблон, по которому будет проверяться вводимое значение, должен заполняться через публичное

свойство. Должна всплывать подсказка (например, ToolTip) с примером правильного ввода (пример должен заполняться через публичный метод).

Публичное свойство для установки и получения введенного значения (set, get):

- при установке: в TextBox вывести значение, если оно соответствует шаблону, либо ничего, в зависимости от того, задан шаблон или нет;
- при получении: если не задан шаблон, то вызывать ошибку (сделать свой класс ошибок). Если введенное значение не соответствует шаблону, выдавать ошибку (сделать свой класс ошибок).

Создать также событие, вызываемое при смене значения.

- Визуальный компонент для ввода значения с проверкой вхождения в диапазон (TextBox, DateTimePicker, NumericUpDown). Должно быть 2 публичных поля для настройки границ диапазона.

Публичное свойство для установки и получения введенного значения (set, get). Если не задан диапазон, то вызывать ошибку (сделать свой класс ошибок)

- при установке: если передаваемое значение не входит в диапазон (а диапазон задан), то не заполнять TextBox, DateTimePicker или NumericUpDown;
- при получении: если не задан диапазон, то вызывать ошибку (сделать свой класс ошибок). Если введенное значение не входит в диапазон, выдавать ошибку (сделать свой класс ошибок).

Создать также событие, вызываемое при смене значения.

Пример. Приведен пример компонента и его свойства в VisualStudio. Для компонента ввода определенного значения в свойствах только получение

его значения (рисунок 1.10а). Для компонента с проверкой по шаблону (в примере компонент для ввода адреса электронной подписи) в свойствах помимо значения еще указывается шаблон (рисунок 1.10б). Для компонента с диапазоном (в примере компонент для ввода числа) в свойствах помимо значения указывается минимальное и максимальное допустимые значения (рисунок 1.10в).

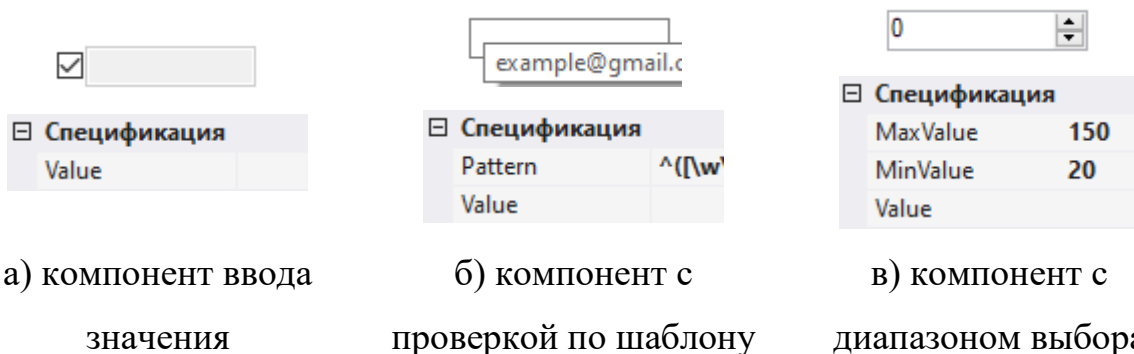


Рисунок 1.10 – Примеры компонент с их особыми настройками

- **Визуальный компонент вывода списков:**

- Визуальный компонент для вывода списка в виде таблицы (DataGridView). DataGridView настроен так, чтобы выбиралась только одна строка и строка целиком, а не ячейка, RowHeaders был скрыт. Отдельный метод для конфигурации столбцов. Через метод указывается сколько колонок в DataGridView добавлять, их заголовки, ширину, признак видимости и имя свойства/поля объекта класса, записи которого будут в таблице выводиться, значение из которого потребуется выводить в ячейке этой колонки. Метод отчистки строк. Публичное свойство для установки и получения индекса выбранной строки (set, get). Публичный параметризованный метод для получения объекта из выбранной строки (создать объект и через рефлекссию заполнить свойства его).

Заполнение может происходить 3 возможными способами, в зависимости от варианта:

- через **параметризованный** метод, у которого в передаваемых параметрах идет список объектов какого-то класса и через этот список идет заполнение DataGridView;
 - через **параметризованный** метод, у которого в передаваемых параметрах идет объект какого-то класса, который добавляется в новую строку DataGridView (в конец таблицы);
 - через **параметризованный** метод, у которого в передаваемых параметрах идет объект какого-то класса, номер строки и номер столбца и требуется заполнить значением из определенного свойства/поля объекта (имя свойства/поля должно быть в конфигурации столбца) ячейку на пересечении строки и столбца (если строк меньше, чем указано, то добавить строки).
- Визуальный компонент для вывода списка в виде списка записей (ListBox). Строки ListBox заполняются на основе макетной строки. В макетной строке указывается некая фраза (например, «Температура воздуха {T}, давление {P}»). Вместо {T} и {P} подставляются значения из свойств/полей объекта какого-то класса. Из каких именно свойств/полей вытаскиваются значения, указывается внутри фигурных скобок (в примере это будут имена свойств/полей T и P). Также задаются символы, которые будут означать начало и конец имени свойства в макетной строке (в примере это будут символы '{' и '}'). Макетная строка и символы начала и конца для имени свойства заполняется через публичный метод. Публичное свойство для установки и получения индекса выбранной строки (set, get). Публичный параметризованный метод для получения объекта из выбранной строки (создать объект и через рефлексия заполнить свойства его).

Заполнение может происходить 3 возможными способами, в зависимости от варианта:

- через **параметризованный** метод, у которого в передаваемых параметрах идет список объектов какого-то класса и через этот список идет заполнение ListBox;
 - через **параметризованный** метод, у которого в передаваемых параметрах идет объект какого-то класса, который добавляется в новую строку ListBox (в конец списка);
 - через **параметризованный** метод, у которого в передаваемых параметрах идет объект какого-то класса, номер строки и имя свойства/поля, значение которого требуется подставить в указанной строке, если этого свойство/поле там еще не заполнено (если строк меньше, чем указано, то добавить строки).
- Визуальный компонент для вывода списка в виде дерева (TreeView). Для дерева задается иерархия (последовательность) из имен свойств/полей какого-то класса. Заполнение дерева происходит по следующему принципу: для объекта класса, который добавляется в дерево, вытаскивается значение (в виде строки) из свойства/поля, которое идет первым в иерархии. Если в корне дерева уже есть ветка с таким названием, то идет переход в нее (если для этого свойства/поля не указано создавать всегда новую ветку), иначе происходит создание новой ветки. Далее вытаскивается значение из второго (указано вторым в иерархии) свойства/поля добавляемого объекта и в ветке ищется подветка с таким названием. Если такая есть и для нее не указано всегда создавать новую ветку, то идет переход в нее, иначе создается новая. И т.д. по всем значениям из иерархии. Иерархия задается через публичный метод. Например, имеется класс «Сотрудник», у

которого имеются поля «ФИО», «Должность» и «Отдел» (могут быть и иные поля, но нас будут интересовать эти). И требуется построить дерево так, чтобы все сотрудники были сгруппированы по отделам и должностям. В таком случае, в дереве заводим иерархию «Отдел», «Должность», «ФИО» и заносим в такое дерево список имеющихся сотрудников так, чтобы для поля «ФИО» всегда создавались новые ветки (на случай, если у нас есть полные однофамильцы). Получается на первом уровне будут указаны отделы, в них должности и в них ФИО сотрудников. Публичное свойство для установки и получения индекса выбранной ветки (конечного элемента одной из ветки) (set, get). Публичный метод для получения выбранной записи (для конечного элемента дерева).

Заполнение может происходить 3 возможными способами, в зависимости от варианта:

- через **параметризованный** метод, у которого в передаваемых параметрах идет список объектов какого-то класса и через этот список идет заполнение TreeView;
- через **параметризованный** метод, у которого в передаваемых параметрах идет объект какого-то класса, который добавляется в TreeView (новая ветка или подветка, если значения свойств уже пересекаются в дереве);
- через **параметризованный** метод, у которого в передаваемых параметрах идет объект какого-то класса, и имя свойства/поля, до которого согласно иерархии, следует сформировать ветки.

Пример. Имеется класс-информация о сотруднике (его идентификатор имя, фамилия, возраст, должность, подразделение, где он работает и стаж). Имеется список сотрудников:

Идент.	Имя	Фамилия	Возр.	Должность	Подразделение	Стаж
1	Иван	Иванов	34	Инженер	Департамент 1	6
2	Петр	Петров	44	Инженер	Департамент 1	12
3	Сергей	Сергеев	55	Руководитель	Департамент 1	34
4	Ольга	Иванова	34	Бухгалтер	Бухгалтерия	8
5	Татьяна	Петрова	44	Ст. бухгалтер	Бухгалтерия	14

Настроим так, чтобы в табличном виде выводилась информация: идентификатор (скрытое поле), фамилия, имя, возраст (рисунок 1.11а). В древовидном виде должна выводиться информация по сотрудникам, сгруппированным по подразделениям и должностям (рисунок 1.11б). В простом списке выводятся информация о том, какой сотрудник как давно работает в компании (рисунок 1.11в).

Фамилия	Имя	Идент.
Иванов	Иван	1
Петров	Петр	2
Сергеев	Сергей	3
Иванова	Ольга	4
Петрова	Татьяна	5

Департамент 1

- Инженер
 - Иванов
 - Петров
- Руководитель
 - Сергеев
- Бухгалтерия
 - Иванова
 - Петрова

Сотрудник Иванов трудится в течении 6 лет
Сотрудник Петров трудится в течении 12 лет
Сотрудник Сергеев трудится в течении 34 лет
Сотрудник Иванова трудится в течении 8 лет
Сотрудник Петрова трудится в течении 14 лет

а) табличный вывод б) вывод в виде дерева в) вывод в виде списка

Рисунок 1.11 – Примеры компонент с выводом данных

Каждому студенту из группы назначается по одному компоненту из каждого из блоков (см. таблицу с вариантами). Получается, каждому нужно будет реализовать 3 компонента: один компонент выбора, один компонент ввода и один компонент вывода списка.

Варианты

ОПИСАНИЕ ФУНКЦИОНАЛА СМОТРИТЕ В ПУНКТЕ «БЛОКИ ВИЗУАЛЬНЫХ КОМПОНЕНТ»

№ вар.	Визуальный компонент выбора	Визуальный компонент ввода	Визуальный компонент вывода списков
1.	Выпадающий список. Список заполняется через метод, передающий список строк	Поле для ввода адреса электронной почты (адрес электронной почты должен соответствовать шаблону)	Список значений. Список заполняется через метод, передающий список объектов
2.	Список с выбором. Список заполняется через метод, передающий список строк	Поле для заполнения даты с проверкой, что дата находится в определенном диапазоне	Дерево значений. Дерево заполняется через метод, передающий список объектов
3.	Список значений. Список заполняется через метод, передающий список строк	Поле для ввода целочисленного значения с возможностью указания значения null	Таблица значений. Таблица заполняется через метод, передающий список объектов
4.	Выпадающий список. Список заполняется через метод, передающий строку	Поле для ввода даты (формат даты должен соответствовать шаблону)	Список значений. Список заполняется через метод, передающий объект
5.	Список с выбором. Список заполняется через метод, передающий строку	Поле для заполнения текста с проверкой, что длина введенного текста находится в определенном диапазоне	Дерево значений. Дерево заполняется через метод, передающий объект

№ вар.	Визуальный компонент выбора	Визуальный компонент ввода	Визуальный компонент вывода списков
6.	Список значений. Список заполняется через метод, передающий строку	Поле для ввода вещественного значения с возможностью указания значения null	Таблица значений. Таблица заполняется через метод, передающий объект
7.	Выпадающий список. Список заполняется через публичное свойство	Поле для ввода номера телефона (номер телефона должен соответствовать шаблону)	Список значений. Список заполняется через метод, передающий объект, номер строки и имя свойства/поля
8.	Список с выбором. Список заполняется через публичное свойство	Поле для заполнения числа с проверкой, что число находится в определенном диапазоне	Дерево значений. Дерево заполняется через метод, передающий объект и имя свойства/поля
9.	Список значений. Список заполняется через публичное свойство	Поле для ввода даты (формат DD.MM.YYYY) с возможностью указания значения null	Таблица значений. Таблица заполняется через метод, передающий объект, номер строки и номер столбца
10.	Выпадающий список. Список заполняется через метод, передающий список строк	Поле для ввода адреса электронной почты (адрес электронной почты должен соответствовать шаблону)	Список значений. Список заполняется через метод, передающий список объектов
11.	Список с выбором. Список заполняется через метод, передающий список строк	Поле для заполнения даты с проверкой, что дата находится в определенном диапазоне	Дерево значений. Дерево заполняется через метод, передающий список объектов

№ вар.	Визуальный компонент выбора	Визуальный компонент ввода	Визуальный компонент вывода списков
12.	Список значений. Список заполняется через метод, передающий список строк	Поле для ввода целочисленного значения с возможностью указания значения null	Таблица значений. Таблица заполняется через метод, передающий список объектов
13.	Выпадающий список. Список заполняется через метод, передающий строку	Поле для ввода даты (формат даты должен соответствовать шаблону)	Список значений. Список заполняется через метод, передающий объект
14.	Список с выбором. Список заполняется через метод, передающий строку	Поле для заполнения текста с проверкой, что длина введенного текста находится в определенном диапазоне	Дерево значений. Дерево заполняется через метод, передающий объект
15.	Список значений. Список заполняется через метод, передающий строку	Поле для ввода вещественного значения с возможностью указания значения null	Таблица значений. Таблица заполняется через метод, передающий объект
16.	Выпадающий список. Список заполняется через публичное свойство	Поле для ввода номера телефона (номер телефона должен соответствовать шаблону)	Список значений. Список заполняется через метод, передающий объект, номер строки и имя свойства/поля
17.	Список с выбором. Список заполняется через публичное свойство	Поле для заполнения числа с проверкой, что число находится в определенном диапазоне	Дерево значений. Дерево заполняется через метод, передающий объект и имя свойства/поля

№ вар.	Визуальный компонент выбора	Визуальный компонент ввода	Визуальный компонент вывода списков
18.	Список значений. Список заполняется через публичное свойство	Поле для ввода даты (формат DD.ММ.YYYY) с возможностью указания значения null	Таблица значений. Таблица заполняется через метод, передающий объект, номер строки и номер столбца
19.	Выпадающий список. Список заполняется через метод, передающий список строк	Поле для ввода адреса электронной почты (адрес электронной почты должен соответствовать шаблону)	Список значений. Список заполняется через метод, передающий список объектов
20.	Список с выбором. Список заполняется через метод, передающий список строк	Поле для заполнения даты с проверкой, что дата находится в определенном диапазоне	Дерево значений. Дерево заполняется через метод, передающий список объектов
21.	Список значений. Список заполняется через метод, передающий список строк	Поле для ввода целочисленного значения с возможностью указания значения null	Таблица значений. Таблица заполняется через метод, передающий список объектов
22.	Выпадающий список. Список заполняется через метод, передающий строку	Поле для ввода даты (формат даты должен соответствовать шаблону)	Список значений. Список заполняется через метод, передающий объект
23.	Список с выбором. Список заполняется через метод, передающий строку	Поле для заполнения текста с проверкой, что длина введенного текста находится в определенном диапазоне	Дерево значений. Дерево заполняется через метод, передающий объект

№ вар.	Визуальный компонент выбора	Визуальный компонент ввода	Визуальный компонент вывода списков
24.	Список значений. Список заполняется через метод, передающий строку	Поле для ввода вещественного значения с возможностью указания значения null	Таблица значений. Таблица заполняется через метод, передающий объект
25.	Выпадающий список. Список заполняется через публичное свойство	Поле для ввода номера телефона (номер телефона должен соответствовать шаблону)	Список значений. Список заполняется через метод, передающий объект, номер строки и имя свойства/поля
26.	Список с выбором. Список заполняется через публичное свойство	Поле для заполнения числа с проверкой, что число находится в определенном диапазоне	Дерево значений. Дерево заполняется через метод, передающий объект и имя свойства/поля
27.	Список значений. Список заполняется через публичное свойство	Поле для ввода даты (формат DD.MM.YYYY) с возможностью указания значения null	Таблица значений. Таблица заполняется через метод, передающий объект, номер строки и номер столбца
28.	Выпадающий список. Список заполняется через метод, передающий список строк	Поле для ввода адреса электронной почты (адрес электронной почты должен соответствовать шаблону)	Список значений. Список заполняется через метод, передающий список объектов
29.	Список с выбором. Список заполняется через метод, передающий список строк	Поле для заполнения даты с проверкой, что дата находится в определенном диапазоне	Дерево значений. Дерево заполняется через метод, передающий список объектов

№ вар.	Визуальный компонент выбора	Визуальный компонент ввода	Визуальный компонент вывода списков
30.	Список значений. Список заполняется через метод, передающий список строк	Поле для ввода целочисленного значения с возможностью указания значения null	Таблица значений. Таблица заполняется через метод, передающий список объектов
31.	Выпадающий список. Список заполняется через метод, передающий строку	Поле для ввода даты (формат даты должен соответствовать шаблону)	Список значений. Список заполняется через метод, передающий объект
32.	Список с выбором. Список заполняется через метод, передающий строку	Поле для заполнения текста с проверкой, что длина введенного текста находится в определенном диапазоне	Дерево значений. Дерево заполняется через метод, передающий объект
33.	Список значений. Список заполняется через метод, передающий строку	Поле для ввода вещественного значения с возможностью указания значения null	Таблица значений. Таблица заполняется через метод, передающий объект
34.	Выпадающий список. Список заполняется через публичное свойство	Поле для ввода номера телефона (номер телефона должен соответствовать шаблону)	Список значений. Список заполняется через метод, передающий объект, номер строки и имя свойства/поля
35.	Список с выбором. Список заполняется через публичное свойство	Поле для заполнения числа с проверкой, что число находится в определенном диапазоне	Дерево значений. Дерево заполняется через метод, передающий объект и имя свойства/поля

№ вар.	Визуальный компонент выбора	Визуальный компонент ввода	Визуальный компонент вывода списков
36.	Список значений. Список заполняется через публичное свойство	Поле для ввода даты (формат DD.MM.YYYY) с возможностью указания значения null	Таблица значений. Таблица заполняется через метод, передающий объект, номер строки и номер столбца